Estruturas de Dados

Prof. Rodrigo Martins
rodrigo.martins@francomontoro.com.br

Cronograma da Aula

- Vetores ou Arrays
- Matrizes ou Arrays Multidimensionais
- Funções
- Ponteiros
- Exemplos e Exercícios

Vetores ou Arrays

- Um array é uma coleção de um ou mais objetos, do mesmo tipo, armazenados em endereços adjacentes de memória. Cada objeto é chamado de elemento do array.

- Da mesma forma que para variáveis simples, damos um nome ao array. O tamanho do array é o seu número de elementos. Cada elemento do array é numerado, usando um inteiro chamado de índice.

- Em C++ , a numeração começa com 0 e aumenta de um em um.

- Assim, o último índice é igual ao número de elementos do array menos um.

Arrays – exemplo1-array.cpp

*exemplo6.cpp

```cpp
#include <iostream>

using namespace std;
#define ESTUDANTES 5

int main()
{
    int indice;
    float total, nota[ESTUDANTES];
    indice = 0;

    //preenche o vetor
    while (indice < ESTUDANTES)
    {
        cout << "Entre a nota do estudante " << indice + 1 << ": ";
        cin >> nota[indice];
        indice = indice + 1;
    }

    cout << "------------------------------" << endl;

    total = 0;
    int qtd = 1;
```

Arrays – exemplo1-array.cpp

```
24      //imprime o vetor
25      for (int i = 0; i < ESTUDANTES; i++)
26      {
27          cout << "Nota " << qtd << ": " << nota[i] << endl;
28          total = total + nota[i];
29          qtd++;
30      }
31
32      cout << endl << "Media: " << total / ESTUDANTES << endl;
33      return 0;
34  }
35
```

Arrays – exemplo2-array.cpp

*exemplo7.cpp

```cpp
#include <iostream>

using namespace std;

#define NOTAS 5

float mediaValoresVetor(int vet[], int tam)
{
    float soma = 0;
    for (int i = 0; i < NOTAS; i++)
    {
        soma += vet[i];
    }
    return soma / tam;
}

int main()
{
    int vet[NOTAS], acima = 0;
    float media;

    for (int i = 0; i < NOTAS; i++)
    {
        cout << "Digite a nota " << i + 1 << ": "  << endl;
        cin >> vet[i];
    }

```

Arrays – exemplo2-array.cpp

```
28      media = mediaValoresVetor(vet, NOTAS);
29      cout << "Media: " << media << endl;
30
31      for (int i = 0; i < NOTAS; i++)
32      {
33          if (vet[i] > media)
34          {
35              acima++;
36          }
37      }
38      cout << "Valores acima da media: " << acima << endl;
39      cout << "Valores abaixo da media: " << NOTAS - acima;
40
41      return 0;
42  }
43
```

Vetores de caracteres exemplo3-array.cpp

*exemplo8.cpp

```cpp
#include <iostream>

using namespace std;

int main()
{
    //char nome[] = {'r', 'o', 'd', 'r', 'i', 'g', 'o', '\0'};
    char nome[] = "rodrigo";
    int i = 0;
    /*
    '\0' é um caracter null, com o valor numérico 0 é considerado false
    Uma string é um array de caracteres, apesar de ser um array,
    deve-se ficar atento para o fato de que as strings têm no elemento
    seguinte a última letra da palavra/frase armazenada, um caractere '\0'.
    */

    //while (nome[i])
    while (nome[i] != '\0')
    {
        cout << nome[i];
        i++;
    }
    return 0;
}
```

Arrays – exemplo4-array.cpp

*exemplo9.cpp

```cpp
#include <iostream>
#include <string.h>

using namespace std;

void inverte(char nome[])
{
    //strlen retorna o tamanho da string
    int tam = strlen(nome);
    for (int i = tam - 1; i >= 0; i--)
    {
        cout << nome[i];
    }
}

```

Arrays – exemplo4-array.cpp

```
16  int main()
17  {
18      char nome[] = "rodrigo";
19      inverte(nome);
20      cout << endl;
21      //isalpha retorna true se caractere testado for alfabético
22      if (isalpha(nome[0]))
23      {
24          cout << "caractere alfabetico" << endl;
25      }
26      else
27      {
28          cout << "caractere numerico" << endl;
29      }
30
31      //isdigit retorna true se for um dígito
32      if (isalpha(nome[0]))
33      {
34          cout << "letra" << endl;
35      }
36      else
37      {
38          cout << "numero" << endl;
39      }
40
41      //isupper retorna true se o caractere for maiusculo
42      if (isupper(nome[0]))
43      {
44          cout << "maiusculo" << endl;
45      }
46      else
47      {
48          cout << "minusculo" << endl;
49      }
50      return 0;
51  }
```

Arrays – exemplo5-array.cpp

*exemplo10.cpp

```cpp
#include <iostream>
using namespace std;

#define TAMANHO 5

int maior(int a[])
{
    int i, max;
    // Achar o maior valor do array
    max = a[0];
    i = 1;
    while (i < TAMANHO)
    {
        if (max < a[i])
        {
            max = a[i];
        }
        i = i + 1;
    }
    return max;
}

```

Arrays – exemplo5-array.cpp

```
23  int main()
24  {
25      int i, valor[TAMANHO];
26      i = 0;
27      while (i < TAMANHO)
28      {
29          cout << "Entre um inteiro: ";
30          cin >> valor[i];
31          i = i + 1;
32      }
33      cout << "O maior eh " << maior(valor) << endl;
34
35      return 0;
36  }
37
```

Matrizes ou Arrays Multidimensionais

- Em C++, é possível também definir arrays com 2 ou mais dimensões.
- Eles são arrays de arrays.
- Um array de duas dimensões podem ser imaginado como uma matriz (ou uma tabela).

Arrays Multidimensionais
exemplo6-array.cpp

*exemplo11.cpp

```cpp
#include <iostream>


using namespace std;

#define LIN 2
#define COL 2


int main()
{
    int matriz[LIN][COL], i, j;

    //preenche a matriz
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 2; j++)
        {
            cout << "Digite um numero inteiro: ";
            cin >> matriz[i][j];
        }
    }

    cout << "========================================================= " << endl;
```

Arrays Multidimensionais
exemplo6-array.cpp

```
25      //imprime a matriz na tela
26      for (i = 0; i < 2; i++)
27      {
28          for (j = 0; j < 2; j++)
29          {
30              cout << "O valor na posicao " << i << " " << j << " eh: "
31                   << matriz[i][j] << endl;
32          }
33      }
34
35      return 0;
36  }
37
```

Exercícios

1. Escreva um programa em C++ que permita a leitura dos nomes de 10 pessoas e armaze os nomes lidos em um vetor. Após isto, o algoritmo deve permitir a leitura de mais 1 nome qualquer de pessoa e depois escrever a mensagem ACHEI, se o nome estiver entre os 10 nomes lidos anteriormente (guardados no vetor), ou NÃO ACHEI caso contrário.
2. Escreva um programa em C++ que permita a leitura das notas de uma turma de 20 alunos. Calcular a média da turma e contar quantos alunos obtiveram nota acima desta média calculada. Escrever a média da turma e o resultado da contagem.
3. Ler um vetor A de 10 números. Após, ler mais um número e guardar em uma variável X. Armazenar em um vetor M o resultado de cada elemento de A multiplicado pelo valor X. Logo após, imprimir o vetor M.

Exercícios

4. Faça um programa em C++ para ler 20 números e armazenar em um vetor. Após a leitura total dos 20 números, o algoritmo deve escrever esses 20 números lidos na ordem inversa.
5. Faça um programa em C++ para ler um valor N qualquer (que será o tamanho dos vetores). Após, ler dois vetores A e B (de tamanho N cada um) e depois armazenar em um terceiro vetor Soma a soma dos elementos do vetor A com os do vetor B (respeitando as mesmas posições) e escrever o vetor Soma.
6. Faça um programa em C++ para ler e armazenar em um vetor a temperatura média de todos os dias do ano. Calcular e escrever:

   a) Menor temperatura do ano

   b) Maior temperatura do ano

   c) Temperatura média anual

   d) O número de dias no ano em que a temperatura foi inferior a média anual

Funções

- É importante lembrar que em C++, uma função é um bloco de código que é definido uma vez e pode ser chamado várias vezes a partir de diferentes partes do programa.

- As funções em C++ podem ter ou não argumentos, e podem ou não retornar valores.

- Para definir uma função em C++, você deve seguir o seguinte formato:

```
tipo_de_retorno nome_da_função (lista de parâmetros) {
        // Corpo da função
}
```

Funções

- Onde
  - **tipo_de_retorno** é o tipo de dado que a função retorna (por exemplo, int, float, double, void, etc.).
  - **nome_da_função** é o nome que você dá para a função.
  - **lista_de_parâmetros** é a lista de argumentos que a função recebe. Cada argumento é composto por um tipo e um nome (por exemplo, int x, float y, double z, etc.).
  - **corpo_da_função** é o bloco de código que contém as instruções que serão executadas quando a função for chamada.

Funções

- Por exemplo, uma função que recebe dois argumentos do tipo int e retorna a soma desses valores seria definida da seguinte forma:

```
int soma(int a, int b) {
    return a + b;
}
```

- Para chamar uma função em C++, basta escrever o nome da função seguido dos argumentos entre parênteses. Por exemplo:

```
int resultado = soma(2, 3);
```

Funções – exemplo1.cpp

```
void nome-da-função (void)
{
        declarações e senteças (corpo da função)
}
```

- O primeiro **void** significa que esta função não tem tipo de retorno (não retorna um valor), e o segundo significa que a função não tem argumentos (ela não precisa de nenhuma informação externa para ser executada).

exemplo1.cpp

```cpp
#include <iostream>

using namespace std;

// DEFINIÇÃO da função alo()
void alo(void)
{
    cout << "Funcao simples" << endl;
}

// Programa Principal
int main()
{
    alo();
}
```

Funções - exemplo2.cpp

exemplo2.cpp

```cpp
#include <iostream>
#include <locale.h>

using namespace std;

//protótipo da função
bool par(int num);
void mensagem();

int main()
{
    /*
     comando de regionalização do C++ para que não somente acentue as palavras
     corretamente, mas que mostre datas e horas em português.*/
    setlocale(LC_ALL, "Portuguese");
    int n = 0;

    mensagem();

    cout << "Digite um número: ";
    cin >> n;

    if (par(n))
    {
        cout << "O numero " << n << " eh par" << endl;
    }
    else
    {
        cout << "O numero " << n << " eh impar" << endl;
    }
    return 0;
}
```

exemplo2.cpp

```
33  bool par(int num)
34  {
35      if (num % 2 == 0)
36      {
37          return true;
38      }
39      else
40      {
41          return false;
42      }
43  }
44
45  void mensagem()
46  {
47      cout << "Aula do Modulo 2" << endl;
48      cout << endl;
49  }
```

Funções que retornam um valor

- Uma função pode retornar um valor para o programa que o chamou.
- Uma função que retorna um valor tem no cabeçalho o nome do tipo do resultado.
- O valor retornado pode ser de qualquer tipo, incluindo, bool, int, float e char.

Funções - exemplo2.1.cpp

```
1   // programa que verifica se 3 numeros podem ser os lados de um
2   // triangulo reto.
3   #include <iostream>
4   #include <locale.h>
5   using namespace std;
6
7   // funcao que calcula o quadrado de um numero
8
9   int quadrado(int n)
10  {
11      return n * n;
12  }
13
```

Funções - exemplo2.1.cpp

```
14  int main()
15  {
16      setlocale(LC_ALL, "Portuguese");
17      int s1, s2, s3;
18      cout << "Entre três inteiros: ";
19      cin >> s1 >> s2 >> s3;
20      // testar com os valores 3, 4 e 5
21      if ( s1 > 0 && s2 > 0 && s3 > 0 &&
22              (quadrado(s1) + quadrado(s2) == quadrado(s3)
23               || quadrado(s2) + quadrado(s3) == quadrado(s1)
24               || quadrado(s3) + quadrado(s1) == quadrado(s2)) )
25      {
26          cout << " " << s1 << " " << s2 << " " << s3 << " podem formar um triângulo reto\n";
27      }
28      else
29      {
30          cout << " " << s1 << " " << s2 << " " << s3 << " não podem formar um triângulo reto\n";
31      }
32  }
```

Funções - exemplo2.2.cpp

exemplo2.2.cpp

```cpp
#include <iostream>
using namespace std;

int obtem_valor()
{
    int valor;
    cout << "Entre um valor: ";
    cin >> valor;
    return valor;
}

int main()
{
    int a, b;
    a = obtem_valor();
    b = obtem_valor();
    cout << "soma = " << a + b << endl;

    return 0;
}
```

Funções - exemplo3.cpp

Considere o programa abaixo que pede ao usuário dois inteiros, armazena-os em duas variáveis, troca seus valores, e os imprime.

exemplo3.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    int n1, n2, temp;
    cout << "Entre com o numero 1: ";
    cin >> n1;
    cout << "Entre com o numero 2: ";
    cin >> n2;
    cout << "Voce entrou com " << n1 << " e " << n2 << endl;
    /* Troca a com b */
    temp = n1;
    n1 = n2;
    n2 = temp;
    cout << "Trocados, eles sao " << n1 << " e " << n2 << endl;
}
```

exemplo3.1.cpp

É possível escrever uma **função** que executa esta operação de troca?

Faça você mesmo?

exemplo3.1.cpp

- Como você já se viu nos exemplos anteriores, em C++ os argumentos são passados por valor.

- Uma vez que somente os valores das variáveis são passados, não é possível para a função **troca()** alterar os valores de a e b porque **troca()** não sabe onde está na memória estas variáveis armazenadas.

- Além disso, **troca()** não poderia ser escrito usando a sentença return porque podemos retornar **APENAS UM** valor (não dois) através da sentença return.

Argumentos passados por referência exemplo3.2.cpp

- A solução para o problema acima é ao invés de passar os valores de n1 e n2, passar uma referência às variáveis n1 e n2.

- Desta forma, **troca()** saberia que endereço de memória escrever, portanto poderia alterar os valores de n1 e n2.

Argumentos passados por referência exemplo3.2.cpp

*exemplo3.2.cpp

```cpp
#include <iostream>
using namespace std;

void troca(int & px, int & py)
{
    int temp;
    temp = px;
    px = py;
    py = temp;
}


int main()
{
    int n1, n2;
    cout << "Entre com o numero 1: ";
    cin >> n1;
    cout << "Entre com o numero 2: ";
    cin >> n2;
    cout << "Voce entrou com " << n1 << " e " << n2 << endl;
    // Troca a com b -- passa argumentos por referencia
    troca(n1, n2);
    cout << "Trocados, eles sao " << n1 << " e " << n2 << endl;
}
```

Argumentos passados por referência

- Quando n1 e n2 são passados como argumentos para troca(), na verdade, somente seus valores são passados.
- A função não podia alterar os valores de n1 e n2 porque ela não conhece os endereços de n1 e n2.
- Mas se referências para n1 e n2 forem passados como argumentos ao invés de n1 e n2, a função troca() seria capaz de alterar seus valores; ela saberia então em que endereço de memória escrever.
- Na verdade, a função não sabe que os endereços de memória são associados com n1 e n2 , mas ela pode modificar o conteúdo destes endereços.
- Portanto, passando uma variável por referência (ao invés do valor da variável), habilitamos a função a alterar o conteúdo destas variáveis na função chamadora.

Outro de passagem por referência exemplo4.cpp

*exemplo4.cpp

```cpp
#include <iostream>

using namespace std;

void altera(int & n1, int & n2)
{
    n1 = 100;
    n2 = 200;

}

int main()
{
    int n1 = 0, n2 = 0;

    cout << "Digite um numero: " << endl;
    cin >> n1;
    cout << "Digite outro numero: " << endl;
    cin >> n2;

    cout << "Primeiro numero: " << n1 << endl;
    cout << "Segundo numero: " << n2 << endl;

    altera(n1, n2);
    cout << "-------------------------------------------------------" << endl;
    cout << "Primeiro numero alterado: " << n1 << endl;
    cout << "Segundo numero alterado: " << n2 << endl;

    return 0;

}
```

Ponteiros

- A memória RAM de um computador é um conjunto de posições adjacentes.
- De uma maneira simplista podemos dizer que ela é um grande vetor e seu índice é formado pelos endereços individuais de memória.
- Os ponteiros em C++ são uma das características mais importantes da linguagem e permitem manipular diretamente a memória do computador.
- Com ponteiros, é possível criar estruturas de dados complexas e executar operações de baixo nível que não seriam possíveis com outros tipos de variáveis.

Ponteiros

- Em C++, um ponteiro é uma variável que armazena o endereço de memória de outra variável.

- Um ponteiro é declarado usando o operador de asterisco *, que é colocado antes do nome da variável. Por exemplo:

```
int* ptr;
```

- Nesse exemplo, ptr é um ponteiro para um valor inteiro.

- Ele não armazena o valor inteiro em si, mas sim o endereço de memória onde o valor está armazenado.

Ponteiros

- Para obter o endereço de memória de uma variável, use o operador de endereço &, que é colocado antes do nome da variável. Por exemplo:

```
int x = 42;
int* ptr = &x;
```

- Nesse exemplo, &x retorna o endereço de memória da variável x e o valor desse endereço é armazenado no ponteiro ptr.

Ponteiros

- Para acessar o valor armazenado em um endereço de memória apontado por um ponteiro, use o operador de de referência *, que é colocado antes do nome do ponteiro. Por exemplo:

```
int y = *ptr;
```

- Nesse exemplo, *ptr retorna o valor armazenado no endereço de memória apontado pelo ponteiro ptr e esse valor é armazenado na variável y.

Ponteiros

- Os ponteiros também podem ser utilizados para criar e manipular estruturas de dados complexas, como listas encadeadas, árvores binárias, grafos, entre outras.

- No entanto, é importante tomar cuidado ao manipular ponteiros, pois eles podem facilmente causar erros de segmentação (também conhecidos como "segfaults"), que ocorrem quando um programa tenta acessar uma área inválida da memória.

Como uma variável é armazenada?

- Imagine que temos este comando em C++:

```
char ch;
```

- O compilador determina um endereço de memória para a variável.

Endereço	Variável	Conteúdo
1000		
1001		
1002	ch	
1003		
1004		
1005		

Como uma variável é armazenada?

- Quando fazemos:

ch = ‘A’;

- estamos dizendo ao compilador para colocar o ‘A’ na posição de memória de ch.

Endereço	Variável	Conteúdo
1000		
1001		
1002	ch	A
1003		
1004		
1005		

Como uma variável é armazenada?

- Imagine agora que nós criássemos um ponteiro ptr.
- Como qualquer outra variável ele ocuparia espaço.

Endereço	Variável	Conteúdo
1000	ptr	
1001		
1002	ch	A
1003		
1004		
1005		

Como uma variável é armazenada?

- Se rodássemos o seguinte comando:

```
ptr = &ch;
```

- Temos o seguinte resultado e podemos dizer que ptr aponta para ch.

Endereço	Variável	Conteúdo
1000	ptr	1002
1001		
1002	ch	A
1003		
1004		
1005		

Declaração de ponteiros

- Devemos ter em mente que um ponteiro é uma variável como outra qualquer, e, por isso, deve ser declarado.

- Para declarar um ponteiro usamos:

```
tipo *ponteiro;
```

- Ex

```
char a, b, *ptr , c, *x;
int v1, *p1;
```

Inicialização de ponteiros

- Todo ponteiro deve ser inicializado, como qualquer variável.
- Um ponteiro zerado nunca pode ser usado, mas podemos inicializa-lo.
- Existe uma constante em C++ que você pode utilizar para inicializar um ponteiro: NULL
- Ela nada mais é que um “apelido” para o número zero.

Inicialização de ponteiros

- Vejamos o código:

```
int a=5, b=7;
int *ptr = NULL;
```

Endereço	Variável	Conteúdo
1000	ptr	NULL
1001		
1002	a	5
1003	b	7
1004		
1005		

Inicialização de ponteiros

- Se agora usarmos:

ptr = &a;

- Agora temos: a → 5 ptr → 1002 *ptr → 5

Endereço	Variável	Conteúdo
1000	ptr	1002
1001		
1002	a	5
1003	b	7
1004		
1005		

Exemplo1-ponteiro.cpp

exemplo1.cpp

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    int var = 10; // declaração de variável padrão
    int *pvar; // declaração de ponteiro
    //sempre inicializar o ponteiro antes de utiliza-lo
    pvar = &var; // &var --> leia como o endereço da variável var
    *pvar = 20; // *pvar muda o valor de var, porque o ponteiro tem o seu endereço
    cout << *pvar << endl; // *pvar mostra o conteúdo da variável apontada pelo ponteiro
    cout << var << endl;
    cout << &*pvar << endl;
    return 0;
}
```

C:\Users\rodri\Google Drive\Uniesi\Estrutura de Dado...

```
20
20
0x67fef8

O Processo retornou 0   tempo de execução : 0.077 s
Pressione uma tecla para continuar...
```

Exemplo2-ponteiro.cpp

exemplo1.1.cpp

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    short usVar = 200;
    long ulVar = 300;
    int iVar = 400;
    cout << "*** Valores e enderecos ***" << endl;
    cout << "usVar: Valor = " << usVar << ", Endereco = " << &usVar << endl;
    cout << "ulVar: Valor = " << ulVar << ", Endereco = " << &ulVar << endl;
    cout << "iVar: Valor = " << iVar << ", Endereco = " << &iVar << endl;
     return 0;
}
```

C:\Users\rodri\Google Drive\Uniesi\Estrutura de Dados\exemplos_Módulo_3\

```
*** Valores e enderecos ***
usVar: Valor = 200, Endereco = 0x67feee
ulVar: Valor = 300, Endereco = 0x67fee8
iVar: Valor = 400, Endereco = 0x67fee4

O Processo retornou 0   tempo de execução : 0.124 s
Pressione uma tecla para continuar...
```

Exemplo3-ponteiro.cpp

exemplo2.cpp

```cpp
#include <iostream>

using namespace std;

//escopo global
int var = 0;

void passagemPorValor(int var){
    var = 20;
}

int main(int argc, char** argv)
{
    var = 10;
    int *pvar;

    pvar = &var;

    passagemPorValor(var);

    cout << var << endl;
    return 0;
}
```

```
10

0 Processo retornou 0   tempo de execução : 0.061 s
Pressione uma tecla para continuar...
```

Exemplo4-ponteiro.cpp

exemplo3.cpp

```cpp
#include <iostream>

using namespace std;


int var = 0;

void passagemPorReferencia(int* n){ // *n aponta para o endereço de pvar
    *n = 20;
}

int main(int argc, char** argv)
{
    var = 10;

    int *pvar;
    pvar = &var;

    passagemPorReferencia(pvar);

    cout << var << endl;
    return 0;
}
```

```
20

O Processo retornou 0   tempo de execução : 0.085 s
Pressione uma tecla para continuar...
```

Exemplo5-ponteiro.cpp

*exemplo4.cpp

```cpp
#include <iostream>

using namespace std;

//exemplo de ponteiro de array

int main(int argc, char** argv)
{
    int array[] = {1,2,3,4,5};
    int* pArray = &array[0];

    cout << *pArray << endl;
    cout << endl;

    for(int i=0;i<5;i++){
        cout << *pArray << endl;
        pArray++;
    }

    return 0;
}
```

```
1

1
2
3
4
5

O Processo retornou 0   tempo de execução : 0.094 s
Pressione uma tecla para continuar...
```

Exercícios

1. Indique verdadeiro ou falso

a) ( ) O operador & permite-nos obter o endereço de uma variável. Permite também obter o endereço de um ponteiro.

b) ( ) Se x é um inteiro e ptr um ponteiro para inteiros e ambos contêm no seu interior o número 100, então x+1 e ptr+1 apresentarão o número 101.

c) ( ) O operador * nos permite obter o endereço de uma variável.

d) ( ) Os ponteiros são variáveis que apontam para endereços na memória.

Correção Exercícios

1. Indique verdadeiro ou falso

a) (V) O operador & permite-nos obter o endereço de uma variável. Permite também obter o endereço de um ponteiro.

b) (F) Se x é um inteiro e ptr um ponteiro para inteiros e ambos contêm no seu interior o número 100, então x+1 e ptr+1 apresentarão o número 101.

c) (F) O operador * nos permite obter o endereço de uma variável.

d) (V) Os ponteiros são variáveis que apontam para endereços na memória.

Exercícios

2. Qual o resultado?

```
cout << a << b << *ptr;
```

3. Se fizermos ptr = &b, qual o resultado?

```
cout << a << b << *ptr;
```

4. Se agora tivermos *ptr = 20, qual o resultado?

Endereço	Variável	Conteúdo
1000	ptr	1002
1001		
1002	a	5
1003	b	7
1004		

Correção Exercícios

2. Qual o resultado?

   cout << a << b << *ptr; 575

3. Se fizermos ptr = &b, qual o resultado?

   cout << a << b << *ptr; 577

4. Se agora tivermos *ptr = 20, qual o resultado? 52020

Endereço	Variável	Conteúdo
1000	ptr	1002
1001		
1002	a	5
1003	b	7
1004		

Exercícios

5. Qual caractere que se coloca na declaração de uma variável para indicar que ela é um ponteiro? Onde se coloca este caractere?

6. O que contém uma variável do tipo ponteiro?

7. Faça um programa em C++ que crie um vetor de 10 inteiros, coloque peça valores ao usuário e depois imprima todos os seus conteúdos na ordem normal e depois inversa. A impressão dos conteúdos deverá ser feita usando ponteiro.

Exercícios

8. Escreva um programa em C++ que crie um array de 5 inteiros, preencha-o com valores fornecidos pelo usuário e, em seguida, calcule a média dos valores usando ponteiros.

Dica: para calcular a média, some todos os valores do array e divida o resultado pelo número de elementos no array.

Exercícios

5. Qual caractere que se coloca na declaração de uma variável para indicar que ela é um ponteiro? Onde se coloca este caractere?

Na declaração de uma variável em C++, o caractere usado para indicar que ela é um ponteiro é o asterisco (*). Esse caractere é colocado imediatamente antes do nome da variável.

6. O que contém uma variável do tipo ponteiro?

Uma variável do tipo ponteiro contém o endereço de memória de outra variável. Ou seja, ao declarar um ponteiro, reservamos um espaço na memória para armazenar um endereço de memória de uma variável do mesmo tipo do ponteiro.

7. Faça um programa em c++ que crie um vetor de 10 inteiros, peça valores ao usuário e depois imprima todos os seus conteúdos na ordem normal e depois inversa. A impressão dos conteúdos deverá ser feita usando ponteiro.

Referência desta aula

- Notas de Aula do Prof. Prof. Armando Luiz N. Delgado

baseado em revisão sobre material de Prof.a Carmem Hara e Prof. Wagner Zola.

- VELOSO, P. et Alli. Estruturas de Dados. Ed. Campus, 1986.

- http://www.cplusplus.com/reference/

Obrigado

Rodrigo